MAC0422 - EP1

Lucas Antonio Pataluch dos Santos
nUSP 13861180

Detalhes da implementação - Geral

- Para preempções, foi utilizado a função pthread_cancel para cancelar cada thread.
- Para todos os escalonadores, a thread worker é a mesma
- Foi criado um struct para cada processo, armazenando suas informações oferecidas pelo arquivo de entrada e necessárias para o funcionamento dos escalonadores
- Foi criado um struct para cada Thread, que armazena informações cruciais para o funcionamento e gerenciamento de cada Thread

Detalhes da implementação - Escalonador com Prioridade

- Funciona na mesma função que o Round Robin, onde, para o escalonador Round Robin, todos os processos possuem prioridade mínima (41) para que o seu quantum sempre seja o mesmo
- A prioridade varia de 1 a 40, com o valor 1 sendo a prioridade mais urgente e a 40, a menos urgente
- Para escalonador por prioridade, o quantum varia de QUANTUM_SEC, para o processo menos urgente, para QUANTUM_SEC * 2, para o mais urgente.

Detalhes da implementação -Escalonador com Prioridade

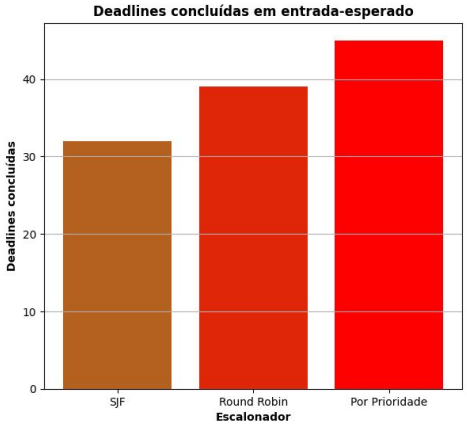
- Funciona na seguinte “pipeline”:
  - Gerencia a fila
  - Para cada thread
    - Se estiver ativa:
      - Se a thread terminou, atualiza as informações da thread e de processos completados
      - Caso contrário, verifica se o quantum já passou para cancelá-la
    - Caso contrário:
      - Pega o primeiro processo na fila, se for RoundRobin, ou pega o primeiro processo de menor prioridade do último processo, se for por prioridade.
      - Define o quantum de acordo com a prioridade: Para RoundRobin, o quantum sempre será o mesmo pois todos os processos sempre possuem prioridade mínima(41)
      - Cria a thread para rodar o processo

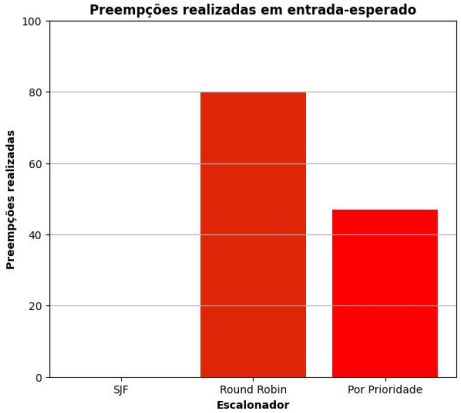
Testes - Computador A

Intel(R) Xeon(R)

CPU E5-2670 0 @ 2.60GHz,

32 CPUS

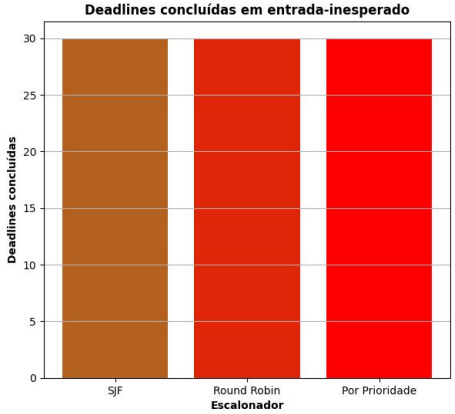


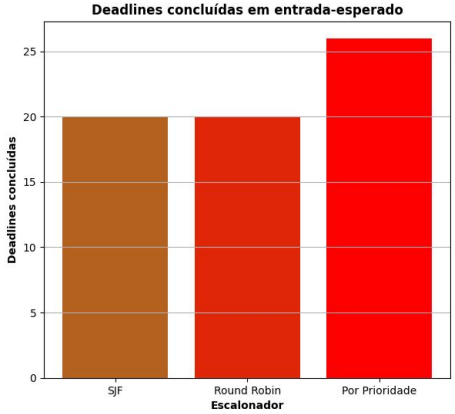


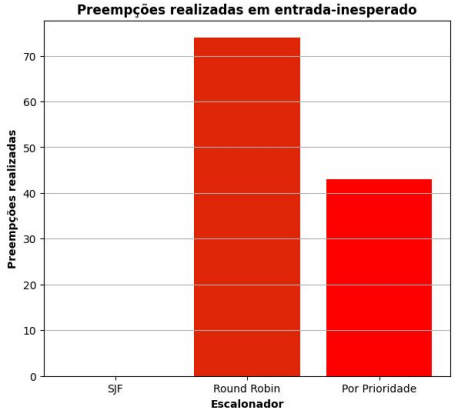
Testes - Computador B

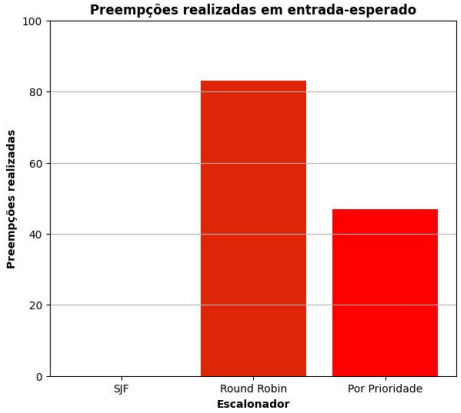
11th Gen Intel(R) Core(TM)

i5-1135G7 @ 2.40GHz, 8CPUS









Testes - entrada-esperado

Esse teste foi projetado para causar um grande estresse em pouco tempo. Tal cenário favorece mais os escalonadores Round Robin e Com Prioridade.

É perceptível que os objetivos do teste, para o computador A, são satisfeitos. Já que, em questão de deadlines cumpridas, o escalonador com prioridade se destaca, logo após o Round Robin e, por último, o sjf. Mesma coisa para o número de preempções, onde o escalonador por prioridade possui menos preempções que o Round Robin.

Já no Computador B, com menos núcleos, essas expectativas não são alcançadas.

Testes - entrada-inesperado

Esse teste foi projetado todos os escalonadores terem, aproximadamente, o mesmo resultado. Desse modo, temos processos (Agora nomeados por gêneros musicais!) que chegam em sequência, mas, também, processos que estão chegam em paralelo à esses, ainda causando briga pelo poder de processamento.

É perceptível que os objetivos do teste, tanto para o computador A quanto para o B, são satisfeitos. Já que, em questão de deadlines cumpridas, todos os escalonadores possuem a mesma quantidade de deadlines cumpridas. Apenas se diferenciam no total de preempções, ainda seguindo o esperado, onde o escalonador com prioridade possui um número menor de preempções do que o Round Robin.

Determinístico ou Não Determinístico?

Após rodar os testes entrada-esperado e entrada-inesperado diversas vezes, temos que os escalonadores são **Não Determinísticos** pois os valores tf e tr de cada processo são alterados toda vez. Somente o número de preempções e o número de deadlines concluídas não se alteram e, desse modo, a média das 30 medições realizadas se mantém no mesmo valor.

Isso se dá por conta da manipulação dos escalonadores com tempo real via pthreads, o que trata atrasos de criação e cancelamento de threads, manipulação de valores, trocas de contexto, processos em background no próprio sistema, etc. Que alteram o tempo absoluto de chegada na fila de cada processo (e consequentemente alterando qual processo é escolhido por qual núcleo) e o tempo absoluto que são finalizados.

Fontes e referências

man page syscalls

https://stackoverflow.com/questions/31149539/finding-the-username-in-a-linux-system-call

https://www.cyberciti.biz/faq/understanding-etcpasswd-file-format/

https://tiswww.case.edu/php/chet/readline/history.html

https://tiswww.case.edu/php/chet/readline/readline.html

https://stackoverflow.com/questions/4770985/how-to-check-if-a-string-starts-with-another-string-in-c

https://chatgpt.com/share/69acec8a-b9f0-800c-99e7-6149050bcc79